

Xavier Harvey
AI Fundamentals
Prof. Tunyan

Due Date: 4 December 2025

Project 4 - Regression ML: Letterboxd Movie Data Analysis

As I've made evident throughout the course of this class, I'm a film enthusiast. Moreover, there exists a contemporary social media application tailor made for fellow film enthusiasts. Said application is Letterboxd, a film-logging platform for users to post when they've seen a film. Given the two directions to take (i.e. regression vs. classification), and my interest in data science, I was compelled to challenge myself with the regression approach.

I. Problem Description

The dataset used in this project, discovered on Kaggle website, is a Letterboxd movie metadata collection containing film information such as titles, release dates (years), descriptions, runtimes, popularity metrics, and a hidden (implied) target variable representing log-likelihood, quantifying user engagement or performance on the Letterboxd platform. The uploaded file comes from a structured Comma-Separated Values dataset (CSV, commonly used for database design), which contains several thousands of films scraped or curated from Letterboxd's open-source movie information.

After initial cleaning and selection the given model uses 6 numerical features derived from the original dataset:

- **year:** release year
- **age_years:** age of the movie (current year - release year)

- **runtime:** movie duration/length in minutes
- **title_length:** number of characters in the film title
- **word_count:** number of words in the movie description/summary/logline
- **title_runtime_ratio:** ratio of title length to runtime

Upon running the code, I ran into an issue with missing numbers (issues with the initial dataset from Kaggle; e.g. movie without a runtime), which were unsupported by Lasso Regression (more on this later). Hence, missing numerical values were handled with median imputation using **SimpleImputer** from **sklearn.imputer**.

While the dataset doesn't directly provide the frequency of a movie being logged, it contains the variable **is_recent**, indicating whether a movie has been logged by users recently. To convert this into a workable, numerical prediction target, I created a new variable:

```
log_likelihood = is_recent.astype(int)
```

The goal of this analysis is to predict the likelihood that a movie has been logged recently on Letterbox, based on its metadata. Given that the target is a numeric likelihood (0 or 1), the task at hand is a regression problem, rather than a classification problem.

II. Train/Test/Split Strategy

The dataset was segmented using an 80/20 train-test split, via the line:

```
train_test_split(X, y, test_size = 0.2, random_state = 42)
```

An 80/20 split, as I've learned, is useful for regression, because it provides enough training data for the models to extract/learn prominent patterns. The remaining 20% serves as a sufficiently large testing set for reliable information, while the random seed 42 (arbitrarily chosen) ensures reproducible results. I didn't use any alternative train-test splits, as the focus was to compare model types rather than tuning split ratios (though I would've loved to explore them).

III. Algorithms Employed

Three regression models were implemented, compared and contrasted: Lasso Regression, Random Forest Regression, and Polynomial Regression. As expected, each method exposes relationships in the metadata in different ways.

Lasso Regression is a linear regression method with L1 regularization. In essence, the model penalizes gargantuan coefficients, with some coefficients shrinking down to exactly zero. This enforces that the model prioritizes the most important features, while mitigating overfitting. Lasso provides a generously simple, and interpretable baseline for comparison, hence why I implemented it first.

Random Forest is an ensemble method (meaning: many small models instead of a single large one) that constructs multiple decision trees and averages their predictions. Each tree perceives different subsets of the data, where all the trees capture nonlinear correlations. The final prediction of the model is the average of all the trees' predictions. This substantially reduces overfitting compared to individual decision trees. Random Forest also provides feature importance scores, showcasing which predicting elements matter most.

Polynomial Regression increases the model's expressiveness by adding polynomial terms, such as:

- x^2, x^3
- Feature combinations

Using **PolynomialFeatures**, aforementioned input features are expanded, with transformed inputs fitted using standard linear regression. This allows the model to learn curved, nonlinear patterns. The main detriments are that higher degrees may cause overfitting, and time complexity rapidly accelerates with compounded degrees. Henceforth, I selected **degree = 3** to showcase an impactful curvature while avoiding extreme complexity.

IV. Results & Analysis

All three models were evaluated using Mean Squared Error (MSE) and Root Mean Squared Error (RMSE). I've made some noteworthy discoveries.

Random Forest Regression performs the best, given that it had the lowest RMSE among the three models. This is to be expected, since Random Forest exposes nonlinear relationships that linear and polynomial models miss.

Polynomial Regression is an improvement on Lasso Regression. Nonlinear terms aid the model in learning recency-based patterns better than a purely linear model. Degree 3 was the ideal choice, as it provided noticeable flexibility, despite possible slight overfitting.

Lasso Regression performed the worst, and it wasn't even close. This model only cooperates with linear relationships, while the target value (i.e log likelihood of recent movie logs) displays nonlinear behavior, limiting performance, thus revealing incompatibility.

Apropos of visualization, I used two key visualizations for my results. The first of which is **Runtime vs. Logging Likelihood**. The corresponding scatter plot contains the x-axis as the runtime, and the y-axis as the log likelihood (0 or 1). This revealed that runtime alone isn't a strong predictor, but displays slight patterns skewed towards mid-length films (approximately 2-2.5 hrs.). The second visualization is **Actual vs. Predicted Plot (Random Forest)**. This compares true values with Random Forest predictions. Based on the plot, Random Forest clusters more closely about the fitting line than the other models would. Additionally, some noise is present, which is reasonable since user logging behavior is only partially explained by metadata.

The typical importance ordering places **age_years** at the zenith, meaning that temporal data is easily the best predictor, meaning that newer films tend to be logged more frequently. Regarding the ancillary attributes, text-based metadata has a modest contribution; ratios and runtimes matter, but much less than release year variables.

Based on the modeling approach and results, the following key findings were discovered:

- Movie recency strongly predicts log likelihood.
- Random Forest Regression is the most effective model for the given dataset.
- Polynomial Regression performs better than Lasso.
- Lasso Regression has the worst performance.
- Text Features (e.g. word count, title length) provide some value, but not much.
- Runtime alone isn't a strong predictor, but becomes useful when operating in conjunction with other features.

Altogether, temporal metadata and nonlinear modeling methods give the most prominent and accurate prediction power for user logging behavior on Letterboxd.

V. Learning Outcomes

Not only did I learn about the underlying metrics used for one of my most used applications, but I learned how potent Machine Learning principles are in the field of data science. At first, I thought the dataset could only be applied with classification, comparing and contrasting user behavior with preferred genres and the like. This was my knee-jerk reaction. However, I realized I was sorely mistaken. Despite my slightly more than negligible background in statistics and probability (even better thanks to this course), I have a stronger grasp on regression principles, and their function in data analytics.

Provided more time, I would optimize the code better with safer preprocessing, resulting in a modified, cleaner dataset. This way, I'd automatically impute values in null spaces, leading to cleaner feature creation. I also found out the the industry standard for ML models use pipelines to avoid data leakage, and employ normalized data for Lasso and Polynomial. Additionally, I'd like to create a Neural Network model using PyTorch or Keras. Maybe someday, I suppose.

VI. Citations

- <https://www.geeksforgeeks.org/machine-learning/a-comprehensive-guide-to-ensemble-learning/>
- <https://machinelearningmastery.com/tour-of-ensemble-learning-algorithms/>
- <https://medium.com/@duygujones/ensemble-learning-combining-models-for-better-machine-learning-d75d7be66b10>
- <https://www.geeksforgeeks.org/machine-learning/linear-regression-python-implementation/>
- <https://www.datacamp.com/tutorial/linear-regression-in-python>
- <https://www.geeksforgeeks.org/machine-learning/python-implementation-of-polynomial-regression/>
- <https://www.geeksforgeeks.org/machine-learning/random-forest-algorithm-in-machine-learning/>
- <https://www.datacamp.com/tutorial/tutorial-lasso-ridge-regression>
- <https://www.geeksforgeeks.org/machine-learning/what-is-lasso-regression/>